

PoRE Lab8 实验报告

姓名：徐锦云 学号：23307130447 耗时：10 小时

PoRE Lab8 实验报告

Task 1: 壳对抗技术实践 (40%)

- 1. 该程序采用了什么加壳技术，你的分析依据是什么？ (20%)
- 2. 如何对该程序进行脱壳（使用的脱壳指令是什么）？ (20%)

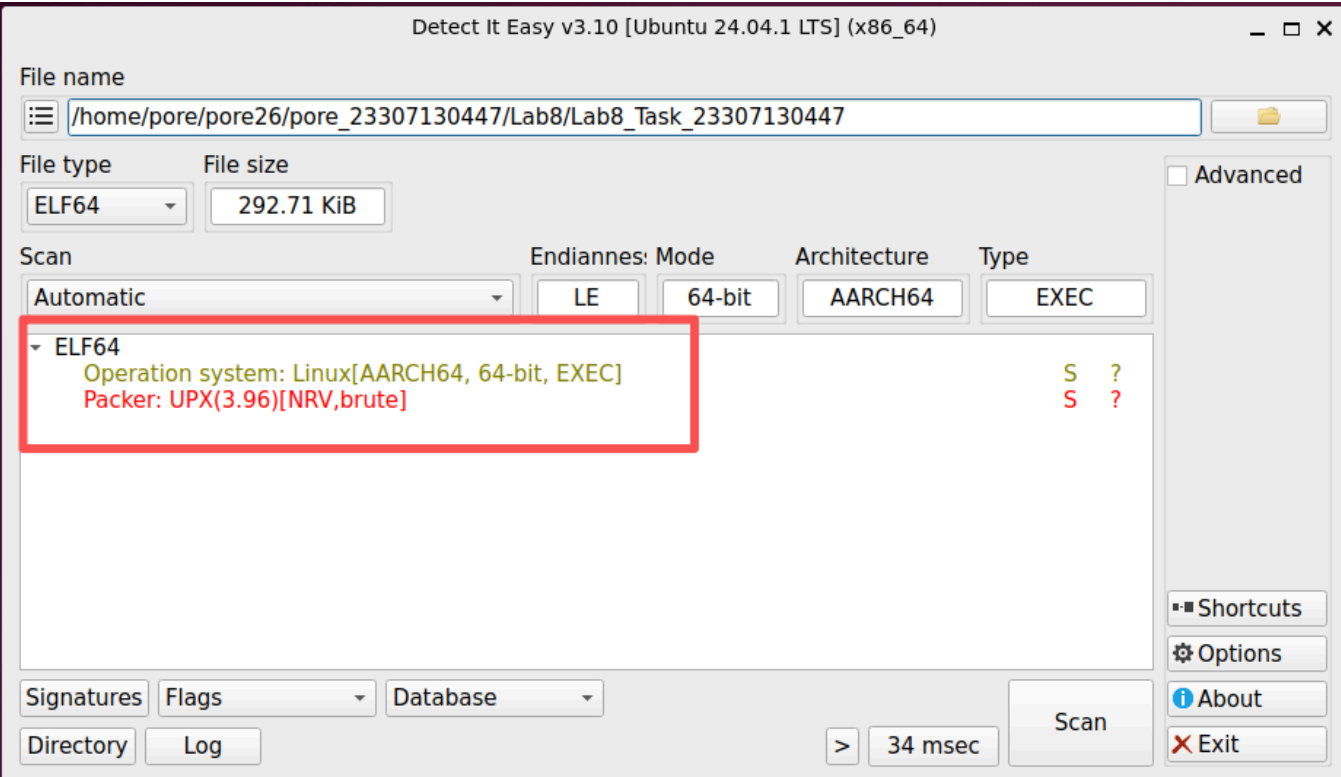
Task 2: 反对逆向工具实践 (60%)

- 1. 这个应用程序采用了什么样的对抗逆向工具技术 (20%)
 - a) 冗余指令 + bl 伪调用 (Redundant Instructions / Spurious BL)
 - b) ret 指令滥用 (Ret Abuse)
 - c) 函数冗余输入 (Redundant Function Arguments)
- 2. 你是如何修改程序，获取到完整程序逻辑的 (20%)
 - 修改前的 main (脱壳后原始版本, 地址 0x4008a4 - 0x4008f4)
 - 修改后的 main (同一地址区间)
 - 修改方法逐条对应
- 3. 该程序的完整功能是什么 (20%)
 - 验证：执行修补后的程序

Task 1: 壳对抗技术实践 (40%)

1. 该程序采用了什么加壳技术，你的分析依据是什么？ (20%)

分析依据：使用 DIE (Detect It Easy v3.10) 对 Lab8_Task_23307130447 进行扫描，结果如下图所示：



DIE 给出的扫描结论是：

字段	值
File type	ELF64
Operation system	Linux (AARCH64, 64-bit, EXEC)
Packer	UPX(3.96)[NRV, brute]

结论：该程序采用 **UPX 3.96** 版本进行加壳，使用 **NRV 压缩算法**，压缩等级为 **brute**。

2. 如何对该程序进行脱壳（使用的脱壳指令是什么）？（20%）

由于加壳工具是 UPX，可直接使用 UPX 自带的 `-d` 选项一键脱壳：

```
$ cp Lab8_Task_23307130447 Lab8_Task_23307130447_unpacked
$ upx -d Lab8_Task_23307130447_unpacked
```

执行结果：

```
# pore @ pore in ~/pore26/pore_23307130447/Lab8 on git:master x [11:15:49] C:2
• $ cp Lab8_Task_23307130447 Lab8_Task_23307130447_unpacked

# pore @ pore in ~/pore26/pore_23307130447/Lab8 on git:master x [11:16:52]
• $ upx -d Lab8_Task_23307130447_unpacked
      Ultimate Packer for eXecutables
      Copyright (C) 1996 - 2020
UPX 3.96      Markus Oberhumer, Laszlo Molnar & John Reiser   Jan 23rd 2020

      File size      Ratio      Format      Name
      -----
      704064 <-      299736      42.57%      linux/arm64      Lab8_Task_23307130447_unpacked

Unpacked 1 file.
```

脱壳前后对比：

```
$ file Lab8_Task_23307130447      # 加壳前
... statically linked, no section header
$ file Lab8_Task_23307130447_unpacked      # 脱壳后
... statically linked, BuildID[sha1]=a1e45917..., for GNU/Linux 3.7.0, not stripped
```

```
# pore @ pore in ~/pore26/pore_23307130447/Lab8 on git:master x [11:17:01]
• $ file Lab8_Task_23307130447
Lab8_Task_23307130447: ELF 64-bit LSB executable, ARM aarch64, version 1 (GNU/Linux), statically linked, no section header

# pore @ pore in ~/pore26/pore_23307130447/Lab8 on git:master x [11:17:44]
• $ file Lab8_Task_23307130447_unpacked
Lab8_Task_23307130447_unpacked: ELF 64-bit LSB executable, ARM aarch64, version 1 (GNU/Linux), statically linked, BuildID[sha1]=a1e45917b2a0182e818daf054b92576fb8661854, for GNU/Linux 3.7.0, not stripped
```

脱壳成功后，原本被加壳的程序节区表、符号表全部恢复，可以使用 Ghidra 进行下一步逆向分析。

Task 2: 反对抗逆向工具实践 (60%)

将脱壳后的 Lab8_Task_23307130447_unpacked 拖入 Ghidra 分析，定位 main 函数（地址 0x400854）。可以发现该程序使用了 **三种** 对抗逆向工具技术，下面分别说明并展示修改前 / 修改后的差异。

1. 这个应用程序采用了什么样的对抗逆向工具技术 (20%)

a) 冗余指令 + bl 伪调用 (Redundant Instructions / Spurious BL)

main 函数在两次 scanf 之后插入了一段毫无作用的指令链：先用 mov x4, x30 保存 LR，然后用 bl 0x4008cc 做“假调用”（紧接着还嵌入了一条无效指令 .word 0xf4327d3e），到了 0x4008cc 又把 LR 还原回来。Ghidra 反编译器会把整段当成一次真实的函数调用，致使 main 反编译结果在这里截断、无法继续向下分析。

```
004008c0 e4 03 1e aa    mov     x4, x30
004008c4 02 00 00 94    bl      FUN_004008cc
004008c8 3e             undefined1 3Eh
004008c9 7d             ??         7Dh    }
004008ca 32             ??         32h    2
004008cb f4             ??         F4h

*****
*                                     *
*                                     *
*****
undefined FUN_004008cc()
w0:1      <RETURN>
FUN_004008cc
XREF[1]:  main:004008c4(c)
004008cc fe 03 04 aa    mov     x30, x4
004008d0 ee 03 1e aa    mov     x14, x30
```

b) ret 指令滥用 (Ret Abuse)

紧接着上一段，程序使用 adr x30, 0x4008ec（把 a2IPyB 的入口地址写入 LR），然后立即 ret。从 CPU 的角度看，ret 只是“跳转到 LR 指向的地址”，所以 **真实的控制流仅仅是跳到 0x4008ec（标号 a2IPyB）继续执行**；但 Ghidra 把 ret 识别为“函数结束”，错误地认为 main 在 0x4008e8 就返回了，于是 main 反编译结果中根本看不到下面真正的分支调度逻辑。

```
004008d8 ca 01 0d ca    eor     x10, x14, x11
004008dc 9e 00 00 10    adr     x30, 0x4008ec
004008e0 1f 20 03 d5    nop
004008e4 1f 20 03 d5    nop
004008e8 c0 03 5f d6    ret

a2IPyB
004008ec 0f 03 00 01    sub     x10, x11, #0x1
004008f0 1f 03 0b 00    nop
```

c) 函数冗余输入 (Redundant Function Arguments)

在 main 调用 aa / bb / cc / dd 之前，程序连续 7 条 mov x0/x2/x3/x4/x5/x6/x7, #0，把所有可能用作 ARM64 调用约定传参的寄存器全部置零。aa / bb / cc / dd 真正只接收 2 个参数（w0, w1），但 Ghidra 看到 8 个寄存器都被写入，就误以为这些函数有 8 个参数，导致反编译出形如 dd(ustack_28, ustack_24, 0, 0, 0, 0, 0, 0) 的错误调用形式，进一步打乱真实参数对应关系。

```

004008a4 00 00 80 d2    mov     x0,#0x0
004008a8 02 00 80 d2    mov     x2,#0x0
004008ac 03 00 80 d2    mov     x3,#0x0
004008b0 04 00 80 d2    mov     x4,#0x0
004008b4 05 00 80 d2    mov     x5,#0x0
004008b8 06 00 80 d2    mov     x6,#0x0
004008bc 07 00 80 d2    mov     x7,#0x0

```

2. 你是如何修改程序，获取到完整程序逻辑的（20%）

操作方式：在 Ghidra 中选中要修改的指令 → 右键 → Patch Instruction → 输入新的指令 → 回车确认；对所有需要置空的指令统一改为 `nop`。

修改前的 main（脱壳后原始版本，地址 0x4008a4 - 0x4008f4）

```

1
2 /* WARNING: Control flow encountered bad instruction data */
3
4 void main(void)
5 {
6     undefined8 extraout_x1;
7     undefined auStack_28 [4];
8     undefined auStack_24 [4];
9     undefined auStack_20 [8];
10    undefined8 local_18;
11
12    local_18 = 0;
13    __isoc99_scanf(&DAT_00464698,auStack_20);
14    __isoc99_scanf("%d %d",auStack_28,auStack_24);
15    FUN_004008cc(0,extraout_x1,0,0,0x4008a4,0,0,0);
16    /* WARNING: Bad instruction - Truncating control flow here */
17    halt_baddata();
18
19

```

； ---- 函数冗余输入：把 x0~x7 全部清零（误导 Ghidra 认为下面要调 8 参数函数） ----

```

4008a4: d2800000    mov     x0, #0x0
4008a8: d2800002    mov     x2, #0x0
4008ac: d2800003    mov     x3, #0x0
4008b0: d2800004    mov     x4, #0x0
4008b4: d2800005    mov     x5, #0x0
4008b8: d2800006    mov     x6, #0x0
4008bc: d2800007    mov     x7, #0x0

```

； ---- 冗余指令 + b1 伪调用：制造假的函数调用 ----

```

4008c0: aa1e03e4    mov     x4, x30                ; 备份 LR
4008c4: 94000002    b1      0x4008cc <main+0x78>   ; "调用"下一条指令
4008c8: f4327d3e    .word   0xf4327d3e             ; 无效指令（永不执行，纯粹混淆反汇编）
4008cc: aa0403fe    mov     x30, x4                ; 还原 LR
4008d0: aa1e03ee    mov     x14, x30
4008d4: aa0e03eb    mov     x11, x14
4008d8: ca0b01ca    eor     x10, x14, x11

```

； ---- ret 指令滥用：通过 adr + ret 伪装成函数返回，实际是跳到 a2IPyB ----

```

4008dc: 1000009e    adr     x30, 0x4008ec <a2IPyB>
4008e0: d503201f    nop
4008e4: d503201f    nop
4008e8: d65f03c0    ret                                ; 实际是 b 0x4008ec

```

```

; ---- a2IPyB 入口处还原 LR 的尾巴 (再次干扰 Ghidra) ----
4008ec: d100056f    sub    x15, x11, #0x1
4008f0: aa0b03eb    mov    x11, x11
4008f4: aa0b03fe    mov    x30, x11
4008f8: b94013e0    ldr    w0, [sp, #16]           ; 真正的逻辑: 读 operation

```

修改后的 main (同一地址区间)

关于函数冗余输入的修改:

```

004008a4 1f 20 03 d5    nop
004008a8 1f 20 03 d5    nop
004008ac 1f 20 03 d5    nop
004008b0 1f 20 03 d5    nop
004008b4 1f 20 03 d5    nop
004008b8 1f 20 03 d5    nop
004008bc 1f 20 03 d5    nop
004008c0 1f 20 03 d5    nop

```

关于冗余指令+b1指令的修改:

```

004008c4 02 00 00 14    b      LAB_004008cc
004008c8 1f             undefined1 1Fh
004008c9 20             ??      20h
004008ca 03             ??      03h
004008cb d5             ??      D5h

                                LAB_004008cc
004008cc 1f 20 03 d5    nop
004008d0 1f 20 03 d5    nop
004008d4 1f 20 03 d5    nop
004008d8 1f 20 03 d5    nop

```

关于ret指令滥用的修改:

```

004008dc 1f 20 03 d5    nop
004008e0 1f 20 03 d5    nop
004008e4 1f 20 03 d5    nop
004008e8 01 00 00 14    b      a2IPyB

                                a2IPyB
004008ec 1f 20 03 d5    nop
004008f0 1f 20 03 d5    nop
004008f4 1f 20 03 d5    nop

```

```

; ---- a) 函数冗余输入: 全部 nop ----
4008a4: d503201f    nop
4008a8: d503201f    nop
4008ac: d503201f    nop
4008b0: d503201f    nop
4008b4: d503201f    nop
4008b8: d503201f    nop
4008bc: d503201f    nop
; ---- b) 冗余指令 + b1 伪调用: b1 改成 b, 其它 nop ----
4008c0: d503201f    nop
4008c4: 14000002    b      0x4008cc <main+0x78>    ; b1 -> b
4008c8: d503201f    nop                        ; 无效指令 -> nop
4008cc: d503201f    nop

```

```
4008d0: d503201f    nop
4008d4: d503201f    nop
4008d8: d503201f    nop
; ---- c) ret 指令滥用: adr + ret 改成 nop + b a2IPyB ----
4008dc: d503201f    nop
4008e0: d503201f    nop
4008e4: d503201f    nop
4008e8: 14000001    b      0x4008ec <a2IPyB>      ; ret -> b
; ---- d) 把 a2IPyB 入口的 3 条干扰指令也清掉 ----
4008ec: d503201f    nop
4008f0: d503201f    nop
4008f4: d503201f    nop
4008f8: b94013e0    ldr     w0, [sp, #16]          ; 真正的逻辑开始: 读 operation
```

修改方法逐条对应

对抗技术	原指令	修改后	说明
a) 函数冗余输入	mov x0/x2/x3/x4/x5/x6/x7, #0 (7 条)	全部 nop	让 Ghidra 看到 aa/bb/cc/dd 仅接收 w0,w1
b) 冗余指令 + bl 伪调用	bl 0x4008cc	b 0x4008cc	直接跳转，不再设置 LR
b) 冗余指令 + bl 伪调用	mov x4,x30 / .word 0xf4327d3e / mov x30,x4 / mov x14,x30 / mov x11,x14 / eor x10,x14,x11	全部 nop	这些指令仅用来欺骗反汇编器
c) ret 指令滥用	adr x30,0x4008ec; nop; nop; ret	nop; nop; nop; b 0x4008ec	让真实控制流可见, Ghidra 不再误判函数结束
c) ret 指令滥用	sub x15,x11,#1; mov x11,x11; mov x30,x11 (a2IPyB 入口)	全部 nop	配套清掉伪装尾巴

修改完成后，再次让 Ghidra 重新反编译 main：

```

1
2 undefined8 main(void)
3
4 {
5     undefined8 uVar1;
6     undefined4 local_28;
7     undefined4 local_24;
8     int local_20;
9     uint local_1c;
10    long local_18;
11
12    local_18 = 0;
13    __isoc99_scanf(&DAT_00464698,&local_20);
14    __isoc99_scanf("%d %d",&local_28,&local_24);
15    if (local_20 == 4) {
16        local_1c = aa(local_28,local_24);
17    }
18    else {
19        if (4 < local_20) {
20LAB_00400980:
21            puts("Invalid choice!");
22            uVar1 = 1;
23            goto LAB_004009a8;
24        }
25        if (local_20 == 3) {
26            local_1c = cc(local_28,local_24);
27        }
28        else {
29            if (3 < local_20) goto LAB_00400980;
30            if (local_20 == 1) {
31                local_1c = bb(local_28,local_24);
32            }
33            else {
34                if (local_20 != 2) goto LAB_00400980;
35                local_1c = dd(local_28,local_24);
36            }
37        }
38    }
39    printf("Result: %d\n",(ulong)local_1c);
40    uVar1 = 0;
41LAB_004009a8:
42    if (local_18 == 0) {
43        return uVar1;
44    }
45    /* WARNING: Subroutine does not return */
46    __stack_chk_fail(&__stack_chk_guard,uVar1,0,local_18);
47}

```

四个被调用的子函数也都能被 Ghidra 正确反编译:

```

1
2 int aa(int param_1,int param_2)
3 {
4     return param_1 + param_2;
5 }

```

```

1
2 int bb(int param_1,int param_2)
3 {
4     return param_1 - param_2;
5 }

```

```

1
2 int cc(int param_1,int param_2)
3 {
4     return param_1 * param_2;
5 }
6 }

```

```

int dd(int param_1,int param_2)
{
    int iVar1;

    if (param_2 == 0) {
        puts("Error: Division by zero!");
        /* WARNING: Subroutine does not return */
        exit(1);
    }
    iVar1 = 0;
    if (param_2 != 0) {
        iVar1 = param_1 / param_2;
    }
    return iVar1;
}

```

3. 该程序的完整功能是什么（20%）

综合 `main` 与四个子函数 `aa/bb/cc/dd` 的逻辑，可以确定该程序是一个 **简单的整数四则运算计算器**：

1. 程序首先从标准输入读入 **三个整数**：`operation`、`num1`、`num2`。
2. 根据 `operation` 的取值，调用不同的子函数：

operation	调用函数	运算	结果
1	bb	减法	<code>num1 - num2</code>
2	dd	除法	<code>num1 / num2</code> ；若 <code>num2 == 0</code> ，打印 <code>Error: Division by zero!</code> 并以退出码 1 退出
3	cc	乘法	<code>num1 × num2</code>
4	aa	加法	<code>num1 + num2</code>
其它	—	—	打印 <code>Invalid choice!</code> 并以退出码 1 退出

3. 将运算结果按格式 `Result: %d\n` 打印到标准输出，正常情况下以退出码 0 退出。

验证：执行修补后的程序

把上述对 `main` 的修补保存为 `Lab8_Task_23307130447_patched`（在 Ghidra 中可以选择 `File → Export Program... → ELF` 导出，也可以用脚本直接打 patch）。在虚拟机里执行：

对四种合法 operation + 异常分支分别测试:

```
$ printf "1\n10 3\n" | ./Lab8_Task_23307130447_patched # 1 → bb (减法)
$ printf "2\n10 5\n" | ./Lab8_Task_23307130447_patched # 2 → dd (除法)
$ printf "3\n6 7\n" | ./Lab8_Task_23307130447_patched # 3 → cc (乘法)
$ printf "4\n8 9\n" | ./Lab8_Task_23307130447_patched # 4 → aa (加法)
$ printf "5\n1 1\n" | ./Lab8_Task_23307130447_patched # 非法 operation
$ printf "2\n10 0\n" | ./Lab8_Task_23307130447_patched # 除零保护
```

```
# pore @ pore in ~/pore26/pore_23307130447/Lab8 on git:master x [11:17:55]
● $ printf "1\n10 3\n" | ./Lab8_Task_23307130447_patched
Result: 7

# pore @ pore in ~/pore26/pore_23307130447/Lab8 on git:master x [11:47:13]
● $ printf "2\n10 5\n" | ./Lab8_Task_23307130447_patched
Result: 2

# pore @ pore in ~/pore26/pore_23307130447/Lab8 on git:master x [11:47:25]
● $ printf "3\n6 7\n" | ./Lab8_Task_23307130447_patched
Result: 42

# pore @ pore in ~/pore26/pore_23307130447/Lab8 on git:master x [11:47:37]
● $ printf "4\n8 9\n" | ./Lab8_Task_23307130447_patched
Result: 17

# pore @ pore in ~/pore26/pore_23307130447/Lab8 on git:master x [11:47:52]
⊗ $ printf "5\n1 1\n" | ./Lab8_Task_23307130447_patched
Invalid choice!

# pore @ pore in ~/pore26/pore_23307130447/Lab8 on git:master x [11:48:03] C:1
⊗ $ printf "2\n10 0\n" | ./Lab8_Task_23307130447_patched
Error: Division by zero!
```

每个用例的实际输出都与上面"修补后伪 C 代码"分析得到的预期完全吻合, 可以确认:

- 程序读入 3 个整数: operation, num1, num2;
- 按 operation $\in \{1, 2, 3, 4\}$ 分别执行 减 / 除 / 乘 / 加, 结果通过 printf("Result: %d\n", ...) 输出;
- 除法分支额外做了被除数为 0 的保护;
- 非法 operation 输入会输出 Invalid choice! 并以退出码 1 终止。

故该程序的完整功能就是一个 **整型四则运算计算器**。